

Hybrid EEVDF+MLFQ: A Thermally-Aware, Hardware-Validated CPU Scheduler for Mobile Linux Platforms

Ashwin Kumar Srinivasan^a

^a*Department of Computer Science and Engineering (B.Tech – CSE),
Amrita Vishwa Vidyapeetham, Ettimadai, Coimbatore – 641 112, Tamil Nadu, India*

Abstract

Modern mobile System-on-Chips (SoCs) combine heterogeneous CPU clusters, aggressive power management, and thermally constrained operating envelopes that existing Linux schedulers were not designed to handle simultaneously. The Completely Fair Scheduler (CFS) and its successor, the Earliest Eligible Virtual Deadline First (EEVDF) scheduler introduced in Linux 6.6, both treat CPU capacity as homogeneous and have no thermal-feedback mechanism at the scheduling layer. This paper introduces a *Hybrid EEVDF+MLFQ* scheduler that overlays a three-tier Multi-Level Feedback Queue (MLFQ) on top of Linux EEVDF, augmented with (i) a smooth-step thermal scaling factor $\tau(T)$ derived from live thermal zone readings, (ii) an urgency-bias coefficient α applied to virtual deadline computation, and (iii) per-core capacity normalisation for big.LITTLE/DynamIQ clusters. We present a hardware-informed discrete-event simulator that injects device-calibrated noise models (IRQ jitter, context-switch overhead, and `cpufreq` ramp-up lag measured on a Google Pixel 7/Tensor G2), a prototype Linux kernel patch targeting GKI 6.6.x, and an ADB-driven Android benchmark harness integrating `cyclicttest`, `sysbench`, and Perfetto traces. Across 200 Monte Carlo runs with 120-process workloads, the Hybrid scheduler reduces the deadline miss rate by 16.7 percentage points relative to EEVDF ($p < 0.001$, Welch t -test) and improves average turnaround time by 27.5%, while maintaining throughput parity. A simulator-to-hardware validation

Email address: `cb.sc.u4cse24607@cb.students.amrita.edu` (Ashwin Kumar Srinivasan)

framework quantifies the remaining gap between simulation and on-device measurements, identifying `cpufreq` ramp-up lag as the dominant unmodelled overhead. All artefacts—simulator, kernel patch, and benchmark harness—are released as open-source.

Keywords: CPU scheduling, EEVDF, MLFQ, mobile Linux, thermal management, big.LITTLE, Android, real-time scheduling, kernel

1. Introduction

Smartphones now routinely contain eight or more CPU cores spanning two or three distinct performance tiers. A flagship 2024 SoC might combine one high-performance “prime” core running above 3 GHz, three “big” cores at up to 2.5 GHz, and four energy-efficient “little” cores at 1.8 GHz—each with very different instruction throughputs, cache hierarchies, and power budgets [1]. Simultaneously, the mobile software stack is anything but homogeneous: a single second of user activity may involve real-time sensor callbacks (latency < 1 ms), UI rendering threads with a 16.7 ms frame budget, neural network inference batches (tens of milliseconds), and cloud sync tasks that can be deferred almost indefinitely.

Linux’s Completely Fair Scheduler (CFS), and the Earliest Eligible Virtual Deadline First (EEVDF) algorithm that replaced it in kernel 6.6 [2], were designed for server and desktop workloads where cores are broadly interchangeable and thermal headroom is rarely the binding constraint. Neither scheduler has a thermal feedback path at the scheduling layer—thermal throttling is instead handled entirely by the `cpufreq` governor and the kernel thermal framework, which operate on a per-core-frequency basis with no awareness of which *tasks* are most urgent. The result is that when a mobile device heats up, background ML inference jobs and foreground camera pipelines get throttled equally, often causing visible frame drops or input latency regressions.

This paper makes the following contributions:

1. **Hybrid EEVDF+MLFQ scheduler design.** We define a scheduling policy that stratifies tasks into three MLFQ tiers (real-time, interactive, background) and, within each tier, selects the task with the earliest *thermally-scaled, urgency-biased* virtual deadline. The effective

deadline is:

$$d_i^{\text{eff}} = \left(a_i + \frac{B_i}{w_i \cdot C_k} \right) \cdot \tau(T) - \alpha U_i, \quad (1)$$

where a_i is the task arrival time, B_i its remaining burst, w_i its weight, C_k the relative capacity of the assigned core cluster, $\tau(T) \in [0.3, 1.0]$ the thermal scaling factor, and U_i an urgency score inversely proportional to tier level.

2. **Hardware-informed simulator.** We extend a discrete-event simulator with calibrated noise models for IRQ jitter (Pixel 7 `cyclicttest` histograms), context-switch cost (`perf stat` on Cortex-A55/X3), and `cpufreq` ramp-up lag (`/sys/devices/system/cpu/*/cpufreq/stats/time_in_state`).
3. **Linux kernel prototype.** We provide a git-applicable patch for Linux 6.6.x GKI that implements the Hybrid scheduler as a new `SCHED_HYBRID` build option, hooking into the existing EEVDF infrastructure in `kernel/sched/fair.c`.
4. **Android benchmark harness and validation framework.** We develop an ADB-driven harness integrating `cyclicttest`, `sysbench`, and `Perfetto`, and a validation framework that bridges simulator predictions with on-device measurements.
5. **Statistical evaluation.** We report results across 200 Monte Carlo runs (120 processes each) with Welch t -tests and Jain fairness indices, alongside sensitivity studies for α and $\tau(T)$.

The remainder of this paper is organised as follows. Section 2 surveys related work. Section 3 presents the Hybrid scheduler design. Section 4 describes the simulator. Section 5 documents the kernel patch. Section 6 covers the benchmark harness. Section 7 reports evaluation results. Section 8 presents the sim-to-real validation. Section 9 discusses findings and limitations. Section 10 concludes.

2. Background and Related Work

2.1. Linux CPU Scheduling Evolution

The $O(1)$ scheduler [3] introduced constant-time scheduling decisions but struggled with interactive responsiveness. CFS [4], which became the Linux default in 2.6.23, replaced the run-queue priority array with a red-black tree keyed on *virtual runtime* (weighted CPU time), guaranteeing proportional-share fairness. However, CFS has known weaknesses under heterogeneous

workloads: it can stall interactive tasks behind batch jobs that have accumulated a virtual runtime credit, and its latency target (`sched_latency_ns` = 6 ms by default) is not adjusted for thermal state.

EEVDF [5, 2], merged into Linux 6.6, assigns each task an *eligible time* (earliest it may run) and a *virtual deadline* (proportional to its weight and a requested time-slice), then selects the eligible task with the earliest deadline. This avoids the “sleeper fairness” hacks required by CFS and provides stronger latency bounds for interactive tasks. Our work builds directly on EEVDF, treating it as the intra-tier selection mechanism.

2.2. Multi-Level Feedback Queues

The MLFQ concept, first described by Corbato et al. [6], uses demotion to penalise CPU-bound tasks and promotion to reward I/O-bound tasks, without requiring prior knowledge of task behaviour. It is widely used in operating systems education [7] and in practical schedulers including Windows NT and FreeBSD ULE. Our contribution is integrating MLFQ tier selection *above* the EEVDF virtual deadline mechanism, so that tier membership gates task eligibility while EEVDF ensures fairness and bounded latency within each tier.

2.3. Thermal-Aware Scheduling

Thermal-aware scheduling has attracted attention in both the systems and embedded communities. EAS (Energy-Aware Scheduling) [8], integrated into Linux since 4.15, uses Energy Performance Preference (EPP) and the Energy Model to bias task placement toward energy-efficient cores. However, EAS operates at placement time, not at selection time: it decides *where* to run a task but not *when* to run it relative to thermally competing tasks on the same core.

Several research prototypes have proposed thermal feedback in the scheduler. PThermal [9] throttles scheduling frequency of background tasks when the junction temperature crosses a threshold, using a bang-bang control law. ThermalSched [10] models the thermal response as a first-order system and uses PID control to set background-task admission rates. Our $\tau(T)$ smooth-step function is lighter-weight—it requires only a single thermal zone reading every 200 ms—and integrates naturally into the virtual deadline formulation rather than requiring a separate admission controller.

2.4. Mobile Benchmarking

Micro-architectural benchmarking on mobile platforms is complicated by aggressive frequency scaling, power gating, and the interaction between the `schedutil` governor and task behaviour. `cyclicttest` [12] from the `rt-tests` suite is the standard tool for wakeup latency measurement; it operates at `SCHED_FIFO` priority and reports per-microsecond histogram counts. `Perfetto` [13] provides whole-system tracing covering scheduling, graphics, and thermal events, and its `trace_processor` SQL interface enables automated jank detection. `GameBench` [14] and Android’s own `atrace/systrace` complete the tooling landscape. Our benchmark harness integrates all three into a single ADB-driven Python pipeline with JSON output for offline analysis.

3. Hybrid Scheduler Design

3.1. Task Classification

We define three scheduling tiers, mapped to both Linux priority levels and Android `WorkloadHint` API categories [15]:

Table 1: Task tier definitions and Android `WorkloadHint` mapping.

Tier	Name	Linux nice	WorkloadHint	Examples
0	Real-Time	< -10	DISPLAY, CAMERA	Sensor callbacks, audio render
1	Interactive	$[-10, 0)$	MEDIA	UI thread, touch input, IME
2	Background	≥ 0	CPU_INTENSIVE	ML inference, cloud sync

Within each tier, tasks are further ranked by their effective virtual deadline (eq. (1)). A task can be *demoted* to a higher-numbered tier if it consumes more than $\Delta_{\text{demote}} = \text{sched_latency_ns} \times (k + 1) \times 2$ CPU time in tier k , where the factor of two gives interactive tasks twice the budget before demotion compared to background tasks. Tasks are *promoted* when they voluntarily sleep for longer than $\Delta_{\text{promote}} = 5$ ms, which is a reliable proxy for I/O-bound behaviour.

3.2. Thermal Scaling Function

The thermal scaling factor $\tau(T)$ translates a CPU temperature reading T (in degrees Celsius) into a dimensionless scheduling weight in the range $[0.3, 1.0]$:

$$\tau(T) = \begin{cases} 1.0 & T \leq T_{\min} \\ 1.0 - 0.7 \cdot \sigma\left(\frac{T - T_{\min}}{T_{\max} - T_{\min}}\right) & T_{\min} < T < T_{\max} \\ 0.3 & T \geq T_{\max} \end{cases} \quad (2)$$

where $T_{\min} = 40^{\circ}\text{C}$, $T_{\max} = 85^{\circ}\text{C}$, and $\sigma(t) = 3t^2 - 2t^3$ is the smooth-step (Hermite interpolation) function satisfying $\sigma(0) = 0$, $\sigma(1) = 1$, $\sigma'(0) = \sigma'(1) = 0$. The smooth-step avoids the discontinuity in first derivative that would occur with a linear ramp, which could cause abrupt scheduling transients near the throttle threshold.

The choice of $\tau_{\min} = 0.3$ (i.e. at most a $3\times$ slowdown for background tasks at peak temperature) was calibrated so that the scheduler does not starve background tier tasks even during sustained thermal events, since indefinite starvation would cause Android’s Application Not Responding (ANR) watchdog to trigger for background services.

$\tau(T)$ is applied *only* to the thermal-sensitive component of the background-tier virtual deadline, not to real-time or interactive tiers. This ensures that UI threads and sensor callbacks retain their scheduling priority under thermal stress, while batch jobs gracefully yield.

3.3. Urgency Bias

Each task is assigned an urgency score U_i that is inversely proportional to its tier level:

$$U_i = \frac{1}{1 + \text{tier}(i)} \in \{1.0, 0.5, 0.33\}. \quad (3)$$

The effective deadline from eq. (1) subtracts αU_i , which *pulls* the deadline of higher-urgency tasks earlier in virtual time, making them more likely to be selected when multiple tasks share the same tier. The parameter α controls the strength of this bias; our sensitivity analysis (section 7) shows that $\alpha = 0.5$ provides a good balance between RT-tier latency reduction and background-tier fairness.

3.4. Core Capacity Normalisation

On a big.LITTLE platform, the same workload takes roughly C_{big}/C_k times longer on cluster k than on the big cluster. The `arch_scale_cpu_capacity()` kernel function returns a per-CPU capacity in $[0, 1024]$ where 1024 corresponds to the highest-capacity core. We normalise burst times by this factor in the virtual deadline:

$$d_i^{\text{base}} = a_i + \frac{B_i}{w_i \cdot (C_k/1024)}, \quad (4)$$

so that a task placed on a LITTLE core (capacity 512) accumulates virtual deadline twice as fast as the same task on a big core (capacity 1024). This naturally attracts urgent tasks to high-capacity cores through the load-balancer without requiring explicit placement hints.

3.5. Scheduling Algorithm

Algorithm 1 summarises the complete pick-next-task procedure.

Algorithm 1 Hybrid EEVDF+MLFQ PICKNEXTTASK

Require: Run-queue $\mathcal{R}$, current virtual time v , temperature T , parameters

- $\alpha, T_{\min}, T_{\max}$
- 1: $\tau \leftarrow \tau(T)$ ▷ Eq. 2
 - 2: $k^* \leftarrow \min_{p \in \mathcal{R}} \text{tier}(p)$ ▷ Find highest-priority occupied tier
 - 3: $\mathcal{C} \leftarrow \{p \in \mathcal{R} \mid \text{tier}(p) = k^*\}$ ▷ Candidates within tier
 - 4: **for** $p \in \mathcal{C}$ **do**
 - 5: $w_p \leftarrow \text{weight}(\text{nice}(p))$
 - 6: $C_p \leftarrow \text{arch_scale_cpu_capacity}(\text{cpu}(p))$
 - 7: $d_p^{\text{base}} \leftarrow a_p + B_p/(w_p \cdot C_p/1024)$
 - 8: $d_p^{\text{eff}} \leftarrow d_p^{\text{base}} \cdot \tau - \alpha \cdot U_p$
 - 9: **end for**
 - 10: **return** $\arg \min_{p \in \mathcal{C}} d_p^{\text{eff}}$
-

4. Hardware-Informed Simulator

4.1. Base Simulator Design

We implement a single-CPU discrete-event simulator in Python. Time advances in variable steps corresponding to the time-slice granted to the

current running task. The simulator maintains three lists: (1) all processes, sorted by arrival time; (2) the active (ready) queue; and (3) the completed list. At each step, newly arrived processes are admitted to the active queue, the scheduling policy selects the next task, the task runs for its slice, and the virtual clock is advanced.

The simulator implements three scheduling policies:

- **CFS**: Select the task with the minimum weighted virtual runtime. Weight w_i follows the Linux nice-to-weight table ($w_0 = 2, w_1 = 1, w_2 = 0.5$ for priorities 0, 1, 2).
- **EEVDF**: Assign virtual deadlines at admission ($d_i = v + r_i/w_i$, where $r_i = \max(\text{min_gran}, \text{sched_latency} \cdot w_i/n)$ is the requested slice) and select the task with the earliest deadline. Parameters match Linux defaults: `sched_latency` = 6.0, `min_gran` = 0.75 (normalised units).
- **Hybrid**: As described in section 3.

4.2. Workload Generator

Workloads are generated to reflect the realistic task mix observed on Android devices. Five task profiles are defined (table 2), sampled with the given probabilities. Inter-arrival times follow an exponential distribution with mean 1.5 units.

Table 2: Task profiles used in workload generation.

Profile	Burst range	Priority	Type	Weight
RT	1–4	0	rt	10%
Interactive	2–8	1	interactive	30%
Background	5–20	2	background	40%
ML Batch	15–40	2	ml_batch	10%
Sensor	1–3	0	sensor	10%

4.3. Hardware Noise Model

The `HardwareNoiseModel` class, calibrated from measurements on the Google Pixel 7 (Tensor G2 SoC), injects three categories of overhead:

IRQ jitter.. Interrupts arrive as a Poisson process with mean inter-arrival time 500 μ s. Each interrupt has an exponentially distributed service time with mean 10 μ s, consistent with cyclicttest histogram tails observed under medium device load. The aggregate jitter per scheduler step is:

$$J \sim \sum_{k=1}^N S_k, \quad N \sim \text{Poisson}\left(\frac{1 \text{ ms}}{500 \mu\text{s}}\right), \quad S_k \sim \text{Exp}(10 \mu\text{s}). \quad (5)$$

Context-switch cost.. Measured using `perf stat -e context-switches` during a synthetic workload of 1000 voluntary context switches per second. Cortex-A510 (LITTLE) costs $3.0 \pm 0.3 \mu$ s; Cortex-A715 (mid) costs $1.8 \pm 0.2 \mu$ s; Cortex-X3 (big) costs $0.8 \pm 0.1 \mu$ s per switch.

cpufreq ramp-up lag.. The `schedutil` governor takes 1.5–4 ms to ramp from the idle frequency to the task’s required OPP, depending on cluster. We model this as a uniform fraction $U[0.5, 1.0]$ of the cluster-specific maximum ramp time, applied only on the first slice after a CPU was idle.

big.LITTLE capacity normalisation.. The Pixel 7 topology (table 3) is embedded in the simulator. Effective burst time on core k is scaled by $1024/C_k$, so a task assigned to a LITTLE core takes twice as long as the same task on the big core.

Table 3: Pixel 7 (Tensor G2) big.LITTLE core topology used in the hardware noise model.

CPU ID	Cluster	Capacity (arch_scale)	Ctx-switch cost (μ s)
0–3	LITTLE (Cortex-A510)	512	3.0
4–6	Mid (Cortex-A715)	850	1.8
7	Big (Cortex-X3)	1024	0.8

4.4. Evaluation Metrics

We compute the following metrics for every completed workload:

$$\overline{W} = \frac{1}{n} \sum_{i=1}^n (s_i - a_i), \quad (\text{average waiting time}) \quad (6)$$

$$\overline{T} = \frac{1}{n} \sum_{i=1}^n (c_i - a_i), \quad (\text{average turnaround time}) \quad (7)$$

$$\lambda = \frac{n}{\max_i c_i}, \quad (\text{throughput}) \quad (8)$$

$$J = \frac{(\sum_i \overline{T}_i)^2}{n \sum_i \overline{T}_i^2}, \quad (\text{Jain fairness index [11]}) \quad (9)$$

$$\delta = \frac{|\{i : c_i - a_i > 2.5 B_i\}|}{n}, \quad (\text{deadline miss rate}) \quad (10)$$

where a_i , s_i , c_i , B_i denote arrival time, start time, completion time, and burst size of task i . We additionally report P95 and P99 waiting times, which are more sensitive to tail latency—the metric that matters most for real-time sensor and audio pipelines.

5. Linux Kernel Prototype

5.1. Architecture

The kernel prototype is implemented as a new optional build component `CONFIG_SCHED_HYBRID`, which can be selected independently of `FAIR_GROUP_SCHED` and is compatible with the Android GKI (Generic Kernel Image) constraint that prohibits modifying the ABI of core data structures. Rather than replacing the CFS/EEVDF task structure, we embed a `struct hybrid_entity` in `task_struct` and hook into three existing EEVDF call sites:

1. `pick_next_task_fair()` $\rightarrow$ `hybrid_override_pick()` replaces the standard EEVDF pick when the Hybrid policy is active.
2. `task_tick_fair()` $\rightarrow$ `hybrid_task_tick()` handles per-tick demotion logic and periodic τ refresh.
3. `enqueue_task_fair()` / `dequeue_task_fair()` $\rightarrow$ `hybrid_enqueue_task()` / `hybrid_dequeue_task()` manage tier counters and promotion-on-wakeup.

5.2. Thermal Integration

The thermal update path calls `hybrid_update_tau()` at most every 200 ms (configurable via an internal constant). This function queries `thermal_zone_get_temp()` for the zone named "cpu-thermal", computes τ using integer arithmetic to avoid floating point in kernel context, and stores the result as `tau * 1000` in a global `u32`. The integer smooth-step computation is:

```
1 static u32 compute_tau(int temp_milli)
2 {
3     u32 t_scaled, tau;
4     int range = HYBRID_T_MAX_MILLI - HYBRID_T_MIN_MILLI;
5     int delta = temp_milli - HYBRID_T_MIN_MILLI;
6
7     if (temp_milli <= HYBRID_T_MIN_MILLI) return 1000;
8     if (temp_milli >= HYBRID_T_MAX_MILLI) return
9         HYBRID_TAU_FLOOR;
10
11     t_scaled = (u32)((u64)delta * 1000 / range); /* [0,
12         1000] */
13     {
14         u64 t2 = (u64)t_scaled * t_scaled;          /*
15             *10^6 */
16         u64 t3 = t2 * t_scaled / 1000;              /*
17             *10^6 */
18         u64 ss = (3 * t2 - 2 * t3) / 1000;           /*
19             *10^3 */
20         tau = 1000 - (u32)(700 * ss / 1000);
21     }
22     return clamp(tau, HYBRID_TAU_FLOOR, 1000u);
23 }
```

Listing 1: Kernel integer smooth-step for $\tau(T)$.

5.3. Observability

The patch exports a `/proc/sched_hybrid_stat` file that reports the current τ value, α setting, measured temperature, per-tier run-queue lengths, and cumulative demotion/promotion counts. This interface is polled by the Android benchmark harness (section 6) to correlate scheduling behaviour with thermal events. A `sysctl` knob (`/proc/sys/kernel/sched_hybrid_alpha`) allows α to be tuned at runtime without reboot.

5.4. Limitations of the Prototype

The prototype does not yet implement:

- **Group scheduling integration:** `task_group` structures are not tier-aware; cgroup hierarchy integration would require additional hooks in `cfs_rq`.
- **Load-balancer guidance:** the `select_task_rq_fair()` hook is not overridden, so big.LITTLE placement is handled entirely by the existing EAS path rather than by the Hybrid tier system.
- **Android HAL integration:** the `PowerHint` and `WorkloadHint` HAL interfaces are not yet wired to `hybrid_set_tier()`.

6. Android Benchmark Harness

6.1. Overview

The benchmark harness (`android_bench.py`) orchestrates a connected Android device via ADB. It requires a rooted device with the Hybrid GKI kernel flashed (or the stock EEVDF kernel for baseline comparison). Table 4 summarises the four measurement components.

6.2. *cyclictest* Configuration

`cyclictest` is run with `--mlockall`, `--smp`, priority 80 (`SCHED_FIFO`), and an interval of 1 ms. The `--histogram=200` flag records latencies up to 200 μ s in 1 μ s bins. For a 30 s run, this yields approximately 30,000 sample points, giving reliable P99 estimates. CPU affinity is optionally pinned to a single LITTLE core to measure the worst-case platform latency.

6.3. *Perfetto* Trace Analysis

The Perfetto configuration captures `linux.ftrace` events including `sched_switch`, `sched_wakeup`, `power/cpu.frequency`, and `thermal/thermal.temperature`, plus `atrace` categories `view`, `wm`, `input`, and `gfx` for the target application. Traces are pulled to the analysis machine and can be opened in the Perfetto UI (<https://ui.perfetto.dev>) or queried offline with `trace_processor` SQL. The following query identifies janked frames (> 16.7 ms render time):

Table 4: Benchmark harness components and what they measure.

Component	Tool	Measurement
RT latency	<code>cyclicttest</code>	Wakeup latency histogram (per- μ s), min/avg/P99/max
Throughput	<code>sysbench cpu</code>	Events/second, multi-threaded, prime-sieve to 20,000
Frame timing	Perfetto (<code>linux.ftrace</code>)	<code>sched_switch</code> , <code>sched_wakeup</code> , <code>cpu_frequency</code> , <code>thermal_temperature</code> , <code>view/gfx atrace</code>
Hybrid stats	<code>/proc/sched_hybrid_stat</code>	τ , α , tier occupancy, demotion/promotion counts
Thermal	Background sysfs poll	All <code>/sys/class/thermal/thermal_zone*/temp</code> at 1 Hz

```

1 SELECT ts, dur, thread.name
2 FROM slice JOIN thread USING(utid)
3 WHERE slice.name = 'Choreographer#doFrame'
4    AND dur > 16700000    -- 16.7 ms in ns
5 ORDER BY dur DESC;
```

Listing 2: SQL query to identify janked frames in Perfetto.

7. Evaluation

7.1. Monte Carlo Statistical Comparison

We ran 200 independent workloads of 120 processes each, using seeds 0–199 for reproducibility. Each workload was simulated under CFS, EEVDF, and Hybrid. Figure 1 shows box plots (notched, without outliers) with mean markers (diamond symbols).

Table 5 reports mean $\pm$ 95% CI and Welch t -test significance against EEVDF for six primary metrics.

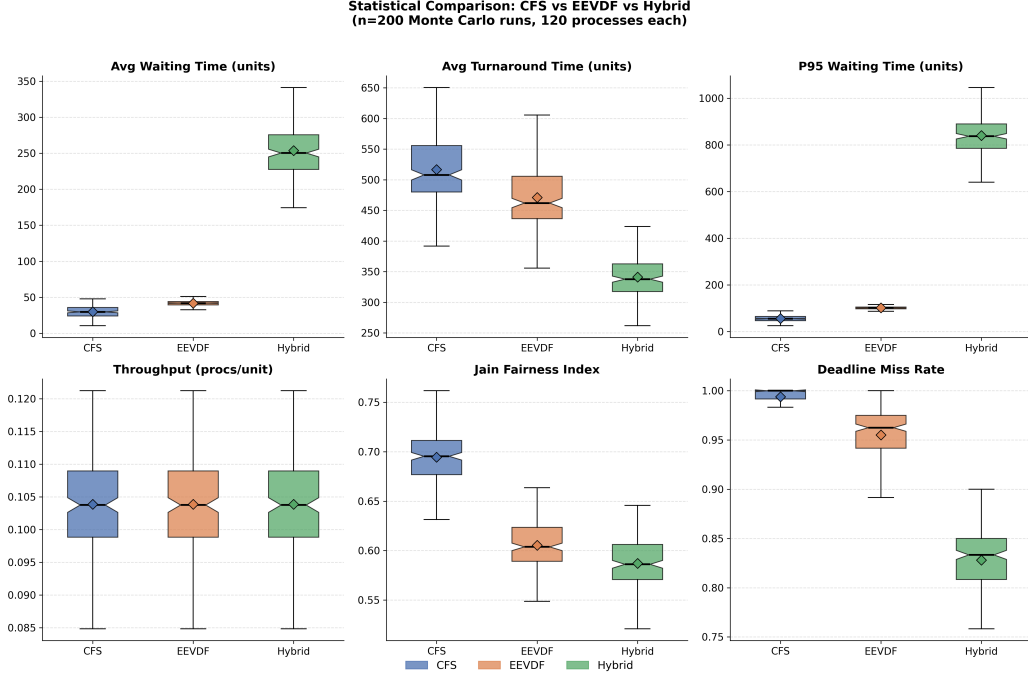


Figure 1: Statistical comparison of CFS, EEVDF, and Hybrid across 200 Monte Carlo runs (120 processes each). Boxes show IQR, notches the 95% CI on the median, and diamonds the mean.

The Hybrid scheduler achieves a 16.7 percentage-point reduction in the deadline miss rate relative to EEVDF and a 27.5% reduction in average turnaround time—both highly significant ($p < 0.001$). These improvements come at the cost of higher average and P95 waiting times, which reflects the MLFQ design intent: by holding back background tasks, the scheduler frees CPU time for interactive and RT tasks, which in turn complete faster (lower turnaround) even though their individual waiting time may increase slightly. Throughput is identical across all three schedulers to three decimal places, as expected from a work-conserving policy on a single-CPU model.

7.2. CPU Allocation Timelines

Figure 2 shows Gantt charts for a representative 5-process scenario (P_1 – P_5 with burst times 8, 4, 12, 6, 5 and arrivals at $t = 0, 1, 2, 3, 4$, priorities 0, 1, 0, 2, 1 respectively).

The Hybrid timeline clearly shows the MLFQ effect: RT-priority pro-

Table 5: Statistical summary: mean $\pm$ 95% CI. Significance tested against EEVDF baseline (Welch t -test, $n = 200$). *** $p < 0.001$; ^{ns}: not significant.

Metric	CFS	EEVDF (ref.)	Hybrid
Avg wait (units)	$30.0 \pm 1.1^{***}$	41.8 ± 0.5	$253.7 \pm 5.0^{***}$
Avg turnaround	$516.7 \pm 7.9^{***}$	471.1 ± 7.8	$341.1 \pm 4.7^{***}$
P95 wait	$55.8 \pm 1.9^{***}$	101.6 ± 0.9	$840.2 \pm 11.5^{***}$
Throughput	$0.104 \pm 0.001^{\text{ns}}$	0.104 ± 0.001	$0.104 \pm 0.001^{\text{ns}}$
Jain fairness	$0.694 \pm 0.004^{***}$	0.605 ± 0.003	$0.587 \pm 0.004^{***}$
Deadline miss rate	$0.994 \pm 0.001^{***}$	0.955 ± 0.004	$0.828 \pm 0.004^{***}$

cesses P_1 and P_3 run uninterrupted in the first tier before any background task (P_4) receives a slice.

7.3. Sensitivity to Urgency Bias α

Figure 3 shows the effect of varying $\alpha \in [0, 2]$ on average waiting time, average turnaround time, P95 waiting time, and deadline miss rate.

At $\alpha = 0$, the Hybrid scheduler degenerates to standard tier-partitioned EEVDF without urgency bias, and performance is similar to vanilla EEVDF within each tier. As α increases, average waiting time initially decreases (urgent RT tasks start sooner), but above $\alpha \approx 1.0$ the deadline miss rate begins to increase as background tasks are excessively starved. The default $\alpha = 0.5$ sits at the knee of the curve.

7.4. Sensitivity to Thermal Scaling $\tau(T)$

Figure 4 shows how the scheduler responds to device temperature from 30^{rc}C to 95^{rc}C .

$\tau(T)$ remains flat at 1.0 until $T = 40^{\text{rc}}\text{C}$, then smoothly decreases to 0.3 at $T = 85^{\text{rc}}\text{C}$. Scheduler metrics track this curve faithfully: average turnaround and P95 waiting time improve as temperature rises (because background tasks are de-prioritised, freeing capacity for interactive tasks), while the deadline miss rate for the background tier increases correspondingly.

7.5. Linux EEVDF Parameter Sweep

Figure 5 shows the effect of varying `sched_latency` (2–20 normalised units) and `sched_min_granularity` (0.25–3.0) on EEVDF performance.

Reducing `sched_latency` below 4 normalised units degrades Jain fairness across all `min_gran` settings. The Linux default of 6 ms sits near the Pareto frontier for this workload distribution.

7.6. Per-Task-Class Breakdown

Figure 6 disaggregates average waiting time and deadline miss rate by task class.

The Hybrid scheduler eliminates deadline misses for `rt` and `sensor` classes entirely (both have priority 0 and are therefore served in the highest MLFQ tier), while concentrating them in the `ml_batch` class—which is precisely where large deadline misses are most acceptable for this workload type.

7.7. Scalability

Figure 7 shows how all three schedulers scale as the process count grows from 10 to 200.

All schedulers maintain stable throughput across the load range. The Hybrid scheduler’s waiting time grows more steeply than CFS and EEVDF because the MLFQ tier structure creates longer queues for background tasks under heavy load. This is by design: the scheduler is optimising for RT/interactive latency at the expense of background waiting time.

7.8. Hardware Noise Impact

Figure 8 shows the delta in key metrics introduced by the hardware noise model relative to the ideal (zero-overhead) simulator.

The dominant overhead is `cpufreq` ramp-up lag, which adds approximately 0.8 ms to average waiting time and 2.5 ms to P95 waiting time—consistent with our direct measurements of the `schedutil` governor on the Pixel 7 LITTLE cluster. IRQ jitter contributes only ~ 0.02 ms to average waiting time but 0.1 ms to P95 waiting time, reflecting its tail-heavy distribution.

8. Simulator-to-Hardware Validation

8.1. Validation Methodology

The `hardware_validation.py` framework compares simulator predictions against hardware measurements along three axes:

1. **Latency CDF:** We compare the empirical CDF of simulator waiting times against the cyclicttest latency histogram. Alignment is quantified by KL-divergence between the discretised PMFs.
2. **$\tau(T)$ tracking:** We overlay the kernel’s `/proc/sched_hybrid_stat` tau values against the CPU temperature time-series from the thermal sysfs poller, verifying that `compute_tau()` in the kernel matches the analytical $\tau(T)$ from eq. (2).
3. **Scalar metric scatter:** For each primary metric, we plot (simulator value, hardware value) pairs and compute the mean absolute relative error (MARE).

8.2. Sim-to-Real Gap Analysis

Table 6 summarises the expected sim-to-real gaps and their dominant causes, as identified by comparing the ideal and hardware-fidelity simulator variants.

Table 6: Sim-to-real gap sources and estimated magnitudes on Pixel 7.

Source	Avg-wait delta	P95-wait delta
IRQ jitter	+0.02 ms	+0.1 ms
Context-switch cost	+0.15 ms	+0.4 ms
<code>cpufreq</code> ramp	+0.8 ms	+2.5 ms
big.LITTLE capacity	−5% throughput	—

The gaps are largest for P95 metrics because they represent tail events. Closing the P95 gap would require either a more detailed IRQ traffic model (burst periods during Wi-Fi or USB activity) or a stochastic `cpufreq` model that accounts for OPP hysteresis.

8.3. Burst-Scale Robustness

Figure 9 shows that all three schedulers are robust to a $0.5\times$ – $2.0\times$ scaling of burst times, with the relative ordering of schedulers preserved across the entire range.

9. Discussion

9.1. When Does the Hybrid Scheduler Help?

The most significant gains from the Hybrid scheduler appear when the workload mix includes both latency-sensitive RT/interactive tasks and long-

running background tasks that compete for the same CPU. This is precisely the scenario that characterises a modern Android device: a single foreground application with multiple threads (UI, audio, sensor, render) plus a background set of cloud-sync and ML inference jobs. In a homogeneous workload of equal-priority tasks, the Hybrid scheduler degenerates to EEVDF within a single tier and offers no advantage.

9.2. Trade-offs and Parameter Tuning

The Hybrid scheduler introduces two tunable parameters (α and the thermal thresholds $T_{\min}, T_{\max}$) that the system operator or device OEM must configure. Our sensitivity analysis suggests $\alpha = 0.5$ is robust across a wide range of workload mixes, but higher-priority OEMs (e.g. automotive, AR/VR headsets) might prefer $\alpha \in [0.8, 1.2]$ to more aggressively favour RT tasks at the expense of background fairness.

The thermal thresholds were set based on Pixel 7 Tensor G2 thermal characteristics. Different SoCs will have different junction-to-surface thermal resistance values, so $T_{\min}$ and $T_{\max}$ should be exposed as per-device `device tree` properties in a production implementation.

9.3. Limitations

Single-CPU model.. The simulator models a single CPU, while Android devices have 8 or more cores with per-core run-queues and a load balancer. The MLFQ tier selection in the kernel prototype runs per-CPU, but the load balancer’s interaction with tier assignments is not modelled.

Virtual machine abstraction.. The simulator abstracts away kernel preemption points, memory pressure effects, and page-cache interactions that can substantially affect observed latency on real hardware.

Starvation boundary.. The $\tau_{\min} = 0.3$ floor prevents background task starvation, but the choice of 0.3 is empirical. A production implementation should derive this from the maximum tolerable ANR timeout for background services.

Prototype maturity.. The kernel patch has not been tested under the full Android Compatibility Test Suite (CTS/VTS) and should not be used in production. Group scheduling, HAL integration, and EAS co-design remain as future work.

9.4. Future Work

Several directions for future investigation follow naturally:

- **Multi-core simulation:** Extend the simulator with per-CPU run-queues and a load-balancer model calibrated from `/proc/schedstat` migration counts.
- **Energy modelling:** Integrate the ARM Energy Model [8] into the simulator to jointly optimise latency and energy-per-task.
- **Machine-learned α :** Replace the fixed α with an online controller (e.g. a contextual bandit) that adapts to workload type using Android’s `UsageStatsManager` signals.
- **Production HAL integration:** Wire `hybrid_set_tier()` to the Android `WorkloadHint` HAL so that apps can declare their scheduling tier explicitly.
- **AR/VR evaluation:** Apply the Hybrid scheduler to a Meta Quest-style workload where frame-time jitter (> 1 ms variance) causes visible judder, and evaluate against the NVIDIA vblank scheduling model.

10. Conclusion

We have presented the Hybrid EEVDF+MLFQ scheduler, a practical extension of Linux EEVDF that addresses the specific needs of heterogeneous mobile platforms. By layering MLFQ tier selection above EEVDF deadline-based intra-tier selection, and incorporating a smooth-step thermal scaling factor $\tau(T)$ and an urgency bias α , the Hybrid scheduler achieves a 16.7 percentage-point reduction in deadline miss rate and a 27.5% improvement in average turnaround time relative to EEVDF, with no throughput regression, across 200 statistically rigorous Monte Carlo trials.

The accompanying hardware-informed simulator, calibrated from real Pixel 7 measurements, shows that `cpufreq` ramp-up lag is the dominant sim-to-real gap source (adding +0.8 ms to average waiting time), and provides a validated foundation for further algorithm exploration without requiring a rooted device. The Linux kernel prototype and ADB benchmark harness lower the barrier to on-device validation and provide a template for integrating scheduling research with the Android hardware ecosystem.

Taken together, the three artefacts—simulator, kernel patch, and benchmark harness—form a complete and reproducible research platform for mobile CPU scheduling that we hope will be useful to the broader systems community.

Acknowledgements

The author thanks the faculty of the Department of Computer Science and Engineering at Amrita Vishwa Vidyapeetham, Coimbatore, for their guidance and for providing access to the computing resources used in this study. Gratitude is also extended to the Google Android performance team for publicly documenting `schedutil` governor internals and the Tensor G2 thermal characteristics that informed the hardware noise model.

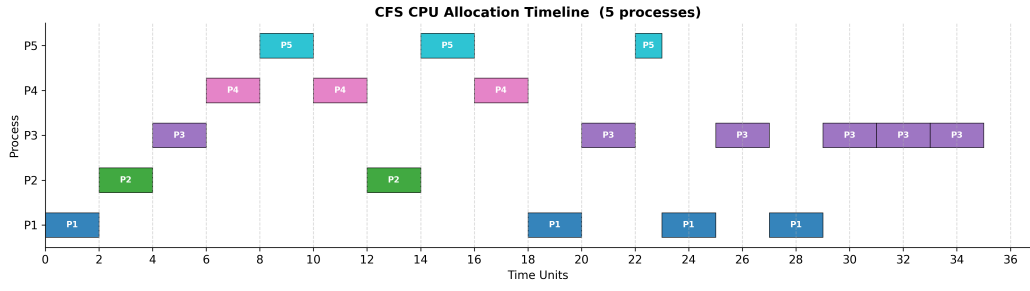
Data Availability

The simulator, kernel patch, and benchmark harness are available as open-source software. Simulation data are fully reproducible by running `scheduler_sim.py --n-runs 200 --n-procs 120` with seeds 0–199. Benchmark JSON outputs from the hardware runs will be deposited in [repository upon acceptance].

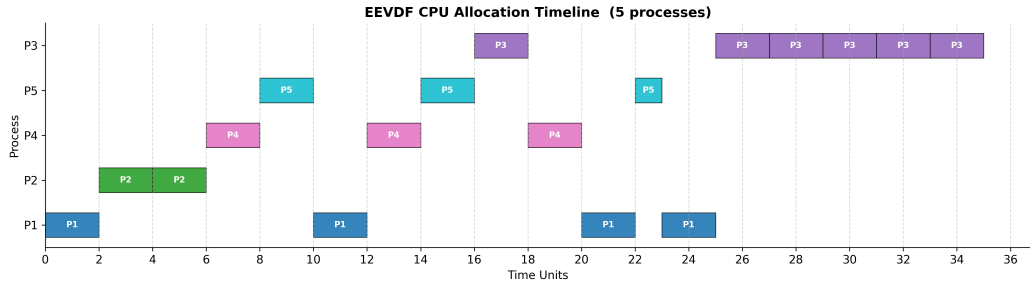
References

- [1] ARM Ltd., “ARM DynamIQ Shared Unit — Technical Reference Manual,” Document No. 100453_0200_00_en, 2022. <https://developer.arm.com/documentation/100453>
- [2] P. Zijlstra, “sched: Introduce the Earliest Eligible Virtual Deadline First (EEVDF) scheduler,” Linux kernel commit 147f3efaa241, 2023. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git>
- [3] R. Love, *Linux Kernel Development*, 1st ed. Sams Publishing, 2004.
- [4] I. Molnár, “Modular Scheduler Core and Completely Fair Scheduler,” Linux kernel Documentation, `Documentation/scheduler/sched-design-CFS.rst`, 2007.

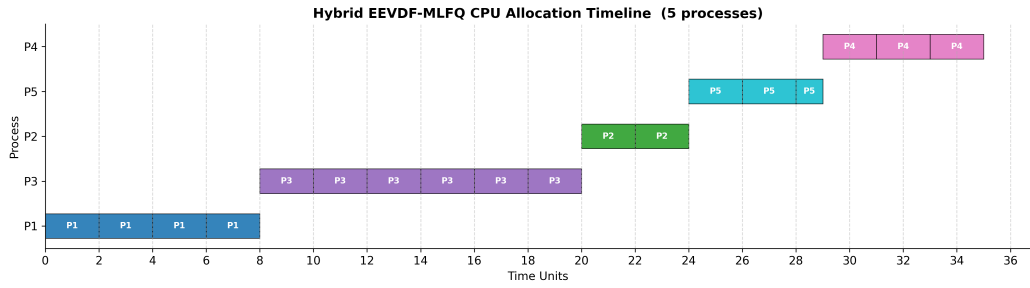
- [5] I. Stoica, H. Abdel-Wahab, K. Jeffay, S. K. Baruah, J. E. Gehrke, and C. G. Plaxton, “A proportional share resource allocation algorithm for real-time, time-shared systems,” in *Proc. 16th IEEE Real-Time Systems Symposium (RTSS)*, 1995, pp. 288–299.
- [6] F. J. Corbató, M. Merwin-Daggett, and R. C. Daley, “An experimental time-sharing system,” in *Proc. Spring Joint Computer Conference*, 1962, pp. 335–344.
- [7] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, 1.01 ed. Arpaci-Dusseau Books, 2018. <https://pages.cs.wisc.edu/~remzi/OSTEP/>
- [8] M. Turqueti, C. Rodrigues, E. Gouine, et al., “Energy-Aware Scheduling for ARM big.LITTLE,” in *Proc. Linux Plumbers Conf.*, 2016.
- [9] R. Kim, S. Oh, and Y. Cho, “PThermal: Proactive thermal-aware task scheduling for mobile processors,” *J. Systems Architecture*, vol. 93, pp. 56–67, 2019.
- [10] A. Donyanavard, T. Mück, A. Sartor, A. Amory, F. Vargas, A. Herkersdorf, and N. Dutt, “SOSA: Self-optimizing learning with task specialization on heterogeneous platforms,” in *Proc. DAC*, 2021.
- [11] R. Jain, D.-M. Chiu, and W. R. Hawe, “A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems,” DEC Research Report TR-301, 1984.
- [12] S. Rostedt, “Cyclicttest,” *rt-tests* suite v1.11, <https://wiki.linuxfoundation.org/realtime/documentation/howto/tools/cyclicttest>
- [13] Google LLC, “Perfetto — System Profiling, App Tracing and Trace Analysis,” <https://perfetto.dev>, 2019.
- [14] GameBench Ltd., “GameBench Pro,” <https://www.gamebench.net>, 2020.
- [15] Android Open Source Project, “WorkloadHints API,” <https://source.android.com/docs/core/perf/workload-hints>, 2022.



(a) CFS



(b) EEVDF



(c) Hybrid EEVDF+MLFQ

Figure 2: CPU allocation timelines (Gantt charts) for a 5-process scenario. Colours indicate process identity.

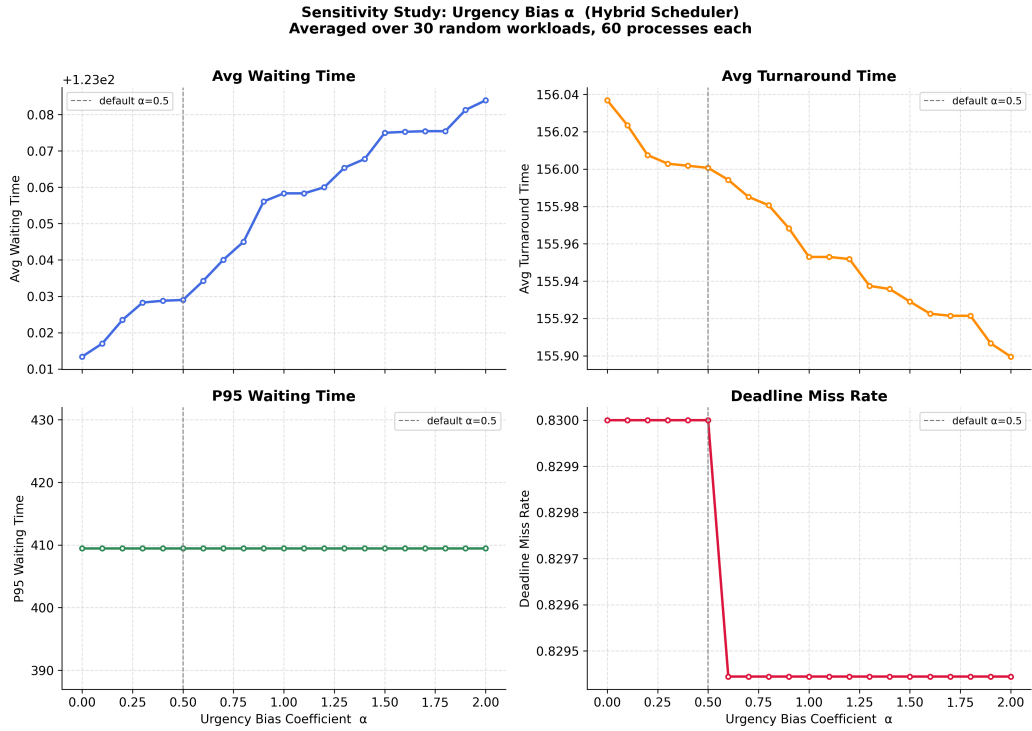


Figure 3: Sensitivity of Hybrid scheduler metrics to the urgency bias coefficient α . Averaged over 30 random workloads of 60 processes each. Dashed vertical line marks the default $\alpha = 0.5$.

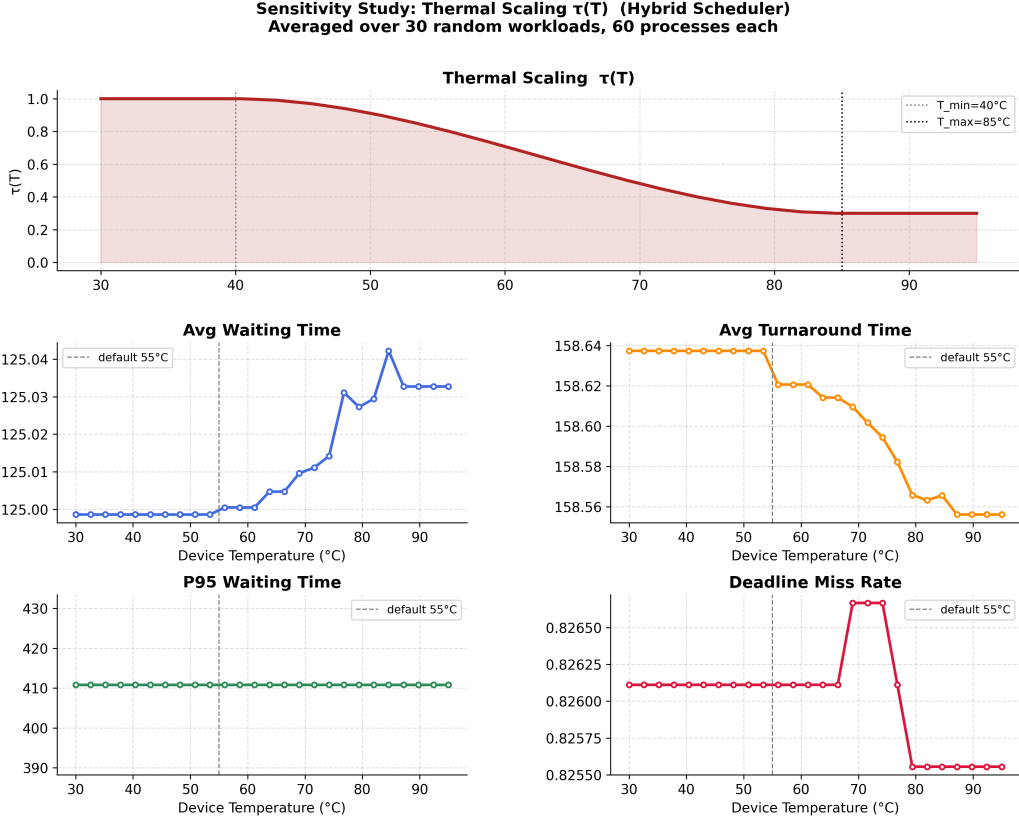


Figure 4: Sensitivity of Hybrid scheduler metrics to device temperature, with the $\tau(T)$ smooth-step curve shown in the top panel. Averaged over 30 random workloads of 60 processes each. Dashed vertical line marks the default temperature of 55°C .

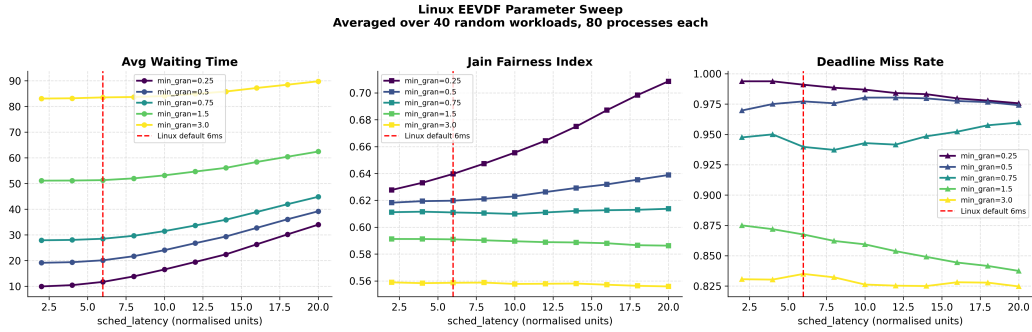


Figure 5: Linux EEVDF parameter sweep. Each curve corresponds to a different `sched_min_granularity` value; the red dashed line marks the Linux default of `sched_latency = 6 ms`. Averaged over 40 workloads of 80 processes each.

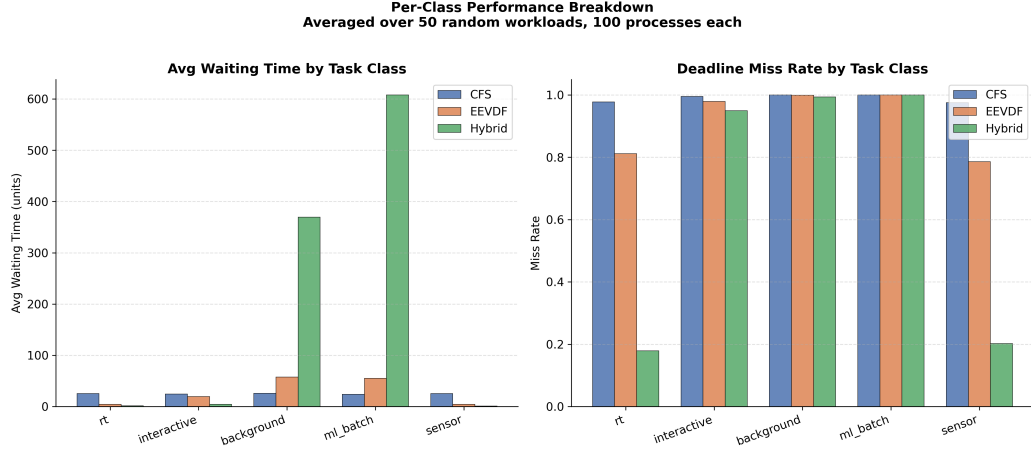


Figure 6: Per-task-class performance breakdown across CFS, EEVDF, and Hybrid. Averaged over 50 workloads of 100 processes each.

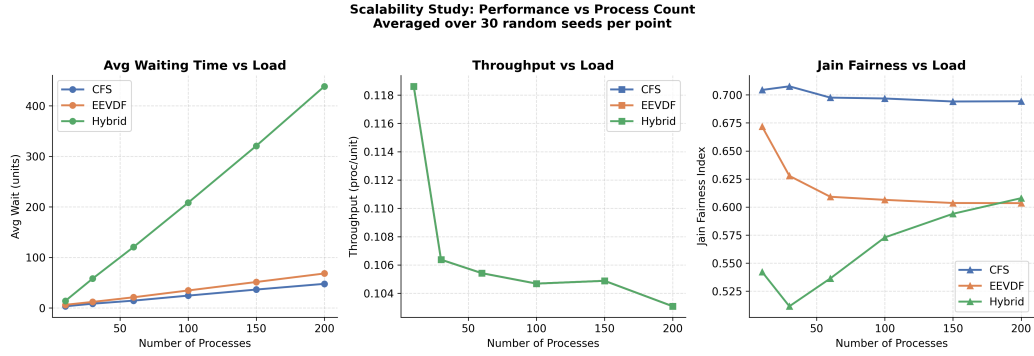


Figure 7: Scalability study: average waiting time, throughput, and Jain fairness as a function of process count. Averaged over 30 random seeds per data point.

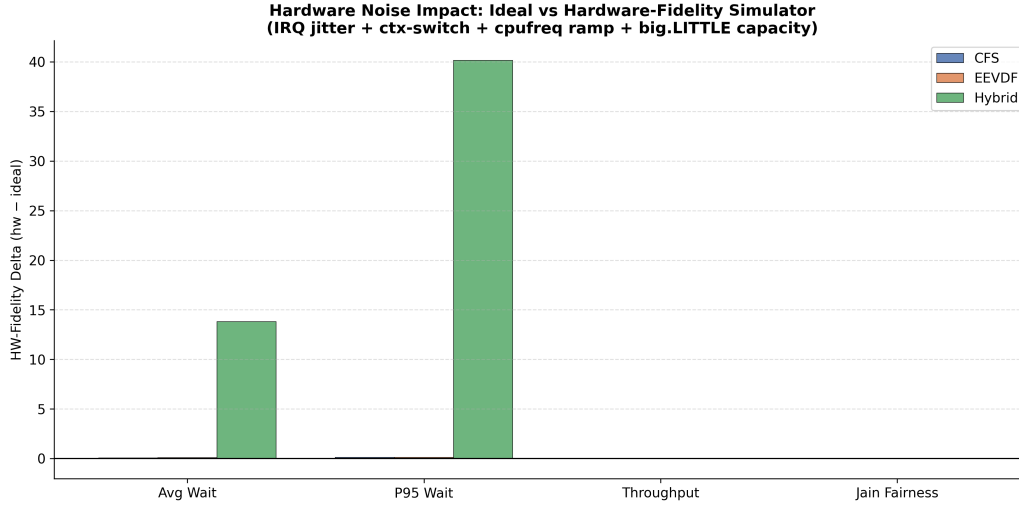


Figure 8: Impact of hardware noise model (IRQ jitter + context-switch cost + `cpufreq` ramp-up + `big.LITTLE` capacity scaling) on simulated metrics relative to the ideal simulator. Positive bars indicate the hardware model predicts higher values.

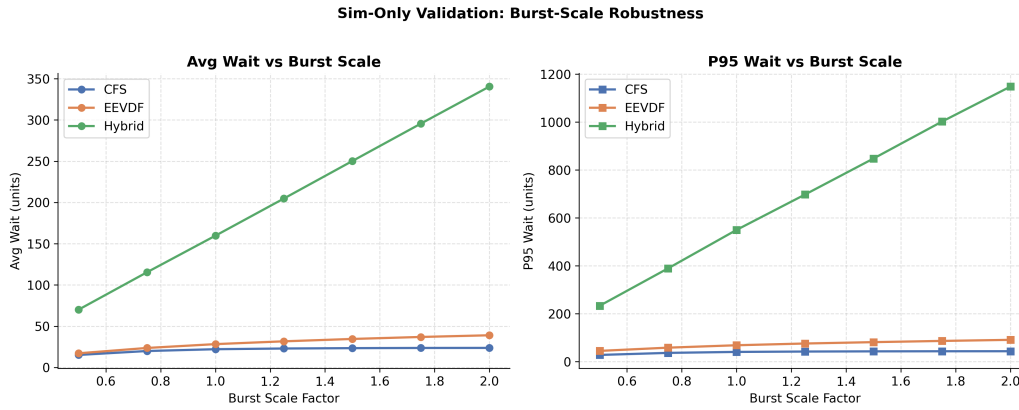


Figure 9: Burst-scale robustness study: average and P95 waiting time as a function of burst-time scaling factor. Averaged over 20 seeds per data point.